

Российский университет дружбы народов имени Патриса Лумумбы

Дисциплина «Имитационное моделирование»

Лабораторная работа № 7

*Дискретно-событийное моделирование: очередь М/М/с и
система Росса*

Выполнила: Куокконен Дарина Андреевна

Группа: НФИбд-01-23

Москва, 2026

Содержание

1. Цель и постановка задачи.
2. Принципы дискретно-событийного моделирования.
3. Среда и организация проекта.
4. Очередь М/М/с: модель, базовый опыт и параметры.
5. Система Росса: процессы, резерв, ремонт и аналитика.
6. Литературные источники и выполненные блокноты.
7. Проверки, ограничения и выводы.

Аннотация. Исследуются две системы с конкуренцией за ограниченные ресурсы: многоканальная очередь и ремонт машин с холодным резервом. Модели реализованы процессами ConcurrentSim, а не дискретизацией времени. Исходные примеры дополнены наблюдением очередей, параметрическими сериями и независимыми аналитическими расчётами. Результаты представлены в CSV, графиках и выполненных Julia-блокнотах. Код четырёх самостоятельных сценариев включён в отчёт из автоматически полученной Quarto-документации.

1. Цель и постановка задачи

Цель работы — освоить процессный подход к дискретно-событийному моделированию и проверить, как ограниченная пропускная способность влияет на ожидание обслуживания и надёжность системы. Выполняется глава 7 «Дискретно-событийное моделирование» практикума А. В. Корольковой и Д. С. Кулябова, стр. 209–219, редакция PDF от 22 мая 2026 года.

В §7.2 требуется реализовать M/M/c в проекте DrWatson и построить необходимые графики. В §7.3 рассматривается система Росса; дополнительные задания предусматривают несколько ремонтников, изменение числа работающих машин, мониторинг загрузки и очереди, графики исправных машин и сравнение с аналитикой. §7.4 распространяет литературное программирование и параметрические варианты на обе модели.

Методичка задаёт два примера, но не назначает имена файлов. В проекте выбраны четыре самостоятельных сценария (Таблица 1). Общие модули, тесты и производные форматы не считаются новыми экспериментами.

Таблица 1: Самостоятельные сценарии лабораторной

Сценарий	Содержание	Объём
<code>mms_queue.jl</code>	Базовая M/M/2	10 клиентов
<code>mms_queue__param.jl</code>	Интенсивность и число каналов	12 × 20 000 клиентов
<code>ross_repair.jl</code>	10 машин, 3 резервные, 1 ремонтник	5 прогонов
<code>ross_repair__param.jl</code>	Число машин и ремонтников	9 × 200 прогонов

2. Принципы дискретно-событийного моделирования

Состояние системы меняется в моменты событий: прибытия заявки, окончания обслуживания, отказа машины или завершения ремонта. Между событиями численности очереди и занятых ресурсов постоянны. Поэтому календарь может сразу перейти к ближайшему событию; малый фиксированный шаг времени не нужен.

В ConcurrentSim объект `Simulation` хранит календарь, `now(env)` возвращает его время, а `@process` запускает процесс. Функция с `@resumable` может приостанавливаться на `@yield` и продолжаться после события. `timeout` задаёт ожидание, `request` запрашивает ресурс, `unlock` освобождает его. В модели ремонта `Store{Process}` хранит резервные процессы, а `interrupt` переводит резервную машину в работу.

Такое представление важно отличать от параллельного вычисления потоками. Одновременное существование процессов модели не означает одновременное исполнение машинных инструкций: события обрабатываются в порядке календаря. Записи с одинаковым временем отражают последовательные стадии одного мгновенного перехода. Они имеют нулевую длительность и не должны исказить интегральные средние.

3. Среда и организация проекта

Вычисления выполнены в отдельном контейнере от пользователя `dakuokkonen`. Каталог проекта — `/workspace/labs/lab07/project`. Julia имеет версию 1.11.9; зависимости перечислены в `Project.toml` и закреплены в `Manifest.toml`. Общий код отделён от сценариев: `src/MMCQueue.jl` описывает очередь, `src/RossRepair.jl` — систему ремонта. `src/experiment_support.jl` содержит сохранение таблиц, графики и расчёт интервалов для независимых повторов.

Таблица 2: Основные зависимости

Пакет	Версия	Назначение
ConcurrentSim	1.5.1	Календарь, процессы и ресурсы
ResumableFunctions	1.0.7	Процессы Julia

Пакет	Версия	Назначение
Distributions	0.25.131	Экспоненциальные времена, t-квантили
StableRNGs	1.0.4	Явные воспроизводимые генераторы
DrWatson	2.19.1	Активация и пути проекта
DataFrames / CSV	1.8.2 / 0.10.17	Таблицы и данные
Plots	1.41.7	Визуализация траекторий
Literate / IJulia	2.21.0 / 1.34.4	Производные форматы и ядро Julia

Проверка среды и пакетов показана на [Рисунок 1](#). Внутри проекта используются обычные команды; основная последовательность сохраняется в Makefile.

```
whoami
pwd
julia --version
julia --project=. -e 'using Pkg; Pkg.status()'
make tangle
make run
make notebooks
make docs
make test
```

```

dakuokkonen@lab07:/workspace/labs/lab07/project$ whoami
dakuokkonen
dakuokkonen@lab07:/workspace/labs/lab07/project$ julia --version
julia version 1.11.9
dakuokkonen@lab07:/workspace/labs/lab07/project$ julia --project=. -e 'using Pkg; Pkg.status()'
Project project v7.0.0
Status `./workspace/labs/lab07/project/Project.toml`
 [336ed68f] CSV v0.10.17
 [6ed1e86c] ConcurrentSim v1.5.1
 [a93c6f00] DataFrames v1.8.2
 [31c24e10] Distributions v0.25.131
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [98b081ad] Literate v2.21.0
 [91a5bcdd] Plots v1.41.7
 [c5292f4c] ResumableFunctions v1.0.7
 [860ef19b] StableRNGs v1.0.4
 [10745b16] Statistics v1.11.4
 [37e2e46d] LinearAlgebra v1.11.0
 [9a3f8284] Random v1.11.0
 [8dfed614] Test v1.11.0
dakuokkonen@lab07:/workspace/labs/lab07/project$

```

Рисунок 1: Пользователь, каталог и установленные пакеты Julia.

4. Очередь М/М/с

4.1 Предположения и стационарная теория

Межприходные интервалы независимы и экспоненциальны с интенсивностью λ , обслуживание в каждом из c одинаковых каналов — с интенсивностью μ . Очередь общая, неограниченная, дисциплина FIFO. Начальное состояние пустое; отказы в обслуживании и уход нетерпеливых клиентов не рассматриваются.

Конструктор `Exponential` принимает **среднее время**, поэтому используются `Exponential(1/λ)` и `Exponential(1/μ)`. При $\lambda=0,9$ и $\mu=0,5$ средний интервал прибытия равен 1,1111, среднее обслуживание — 2 единицам времени.

Пусть $a=\lambda/\mu$ и $\rho=\lambda/(c\mu)$. Стационарный режим существует только при $\rho<1$. Вероятность пустой системы и вероятность ожидания по формуле Эрланга С:

$$p_0 = \left[\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!(1-\rho)} \right]^{-1}, P_{wait} = \frac{a^c p_0}{c!(1-\rho)}.$$

Характеристики очереди и системы вычисляются как

$$L_q = \frac{\rho P_{wait}}{1-\rho}, W_q = \frac{L_q}{\lambda}, W = W_q + \frac{1}{\mu}, L = \lambda W.$$

При $c=1$ они сводятся к М/М/1, например $W=1/(\mu-\lambda)$. Это используется как независимая проверка реализации формулы. При $\rho \geq 1$ программа возвращает для стационарных характеристик `missing`, а не нули или отрицательные числа.

4.2 Процесс заявки и наблюдение событий

Все моменты прибытия заранее получают накоплением экспоненциальных интервалов, как в примере практикума. Затем для каждого клиента создаётся процесс. До получения ресурса клиент относится к очереди, после получения — к занятым каналам. Время обслуживания выбирается при начале обслуживания. Такая последовательность обращений к генератору сохраняет смысл исходного примера и делает `seed` однозначным.

В листинге 1 приведена центральная часть функции `customer` из `src/MMCQueue.jl`. `record!` сохраняет новое состояние и текущее время.

```
@yield timeout(env, arrival)
monitor.queue += 1
record!(monitor, env, "arrival", id)
verbose && println("Customer $id arrived: ", now(env))
@yield request(server)
monitor.queue -= 1
monitor.busy += 1
starts[id] = now(env)
record!(monitor, env, "service_start", id)
verbose && println("Customer $id entered service: ", now(env))
@yield timeout(env, rand(rng, service))
ends[id] = now(env)
monitor.busy -= 1
record!(monitor, env, "departure", id)
@yield unlock(server)
verbose && println("Customer $id exited service: ", now(env))
```

Листинг 1 — События одной заявки, `src/MMCQueue.jl`.

Для клиента сохраняются прибытие, начало и окончание обслуживания, ожидание, длительность обслуживания и пребывание в системе. Для календаря — длина очереди, число занятых каналов и общее число клиентов. После полного исчерпания календаря `run(env)` может перевести его часы на горизонт `Inf`; поэтому физическое время завершения опыта определяется последним фактическим уходом, `maximum(ends)`, а не значением пустого календаря.

4.3 Базовый опыт: десять клиентов

Сохранены исходные условия: 10 клиентов, $c=2$, $\lambda=0,9$, $\mu=0,5$, `StableRNG(123)`.

Команда запуска:

```
julia --project=. scripts/mmc_queue.jl
```

Ниже непосредственно включён результат Literate для этого сценария. Фрагмент содержит действительный код получения CSV, мониторинга и графиков.

4.3.1 Очередь М/М/с: десять клиентов

Базовый пример §7.2.4–7.2.5, стр. 214–215. Заявки поступают в общую FIFO-очередь.

```
using DrWatson
@quickactivate "project"
include(sourcedir("MMCQueue.jl"))
using .MMCQueue
include(sourcedir("experiment_support.jl"))
```

4.3.1.1 Исходные параметры и события

Интенсивности: $\lambda=.9$, $\mu=.5$; среднее обслуживание равно двум единицам времени. Моменты прибытия генерируются до запуска календаря, как в примере практикума.

```
result = simulate_mmc(num_customers=10, num_servers=2,  $\lambda=.9$ ,  $\mu=.5$ ,
seed=123, verbose=true)
save_table("mmc_customers.csv", result.customers)
save_table("mmc_events.csv", result.events)
result.customers
```

4.3.1.2 Наблюдаемые характеристики короткого прогона

Это запуск из пустого состояния, а не точная оценка стационарного режима.

```
observed = DataFrame([queue_summary(result)])
save_table("mmc_summary.csv", observed)
observed
```

4.3.1.3 Очередь и занятые каналы

```
p_events = queue_plot(result.events; title="M/M/2: 10 customers,
lambda=0.9, mu=0.5")
savefig(p_events, imagepath("01-plot.png"))
display(p_events)
```


4.3.1.4 Время ожидания и обслуживания каждого клиента

```
p_clients = customer_timeline(result.customers)
savefig(p_clients, imagepath("02-plot.png"))
display(p_clients)
```

4.3.1.5 Аналитический ориентир

$\rho=0.9 < 1$: стационарные формулы существуют. Для содержательного сравнения используется длинная параметрическая серия, а не только эти десять клиентов.

```
theory = DataFrame([erlang_c(.9, .5, 2)])
save_table("mmc_theory.csv", theory)
theory
```

Первый клиент прибывает в 0,145185, последний — в 7,828990. Последнее обслуживание завершается в 18,690265. Все десять клиентов обслужены, восемь ожидали свободный канал. Среднее ожидание равно 4,856796, пребывание в системе — 8,410492. Наблюдаемое среднее обслуживание 3,553696 отличается от математического ожидания 2: выборка из десяти случайных величин мала.

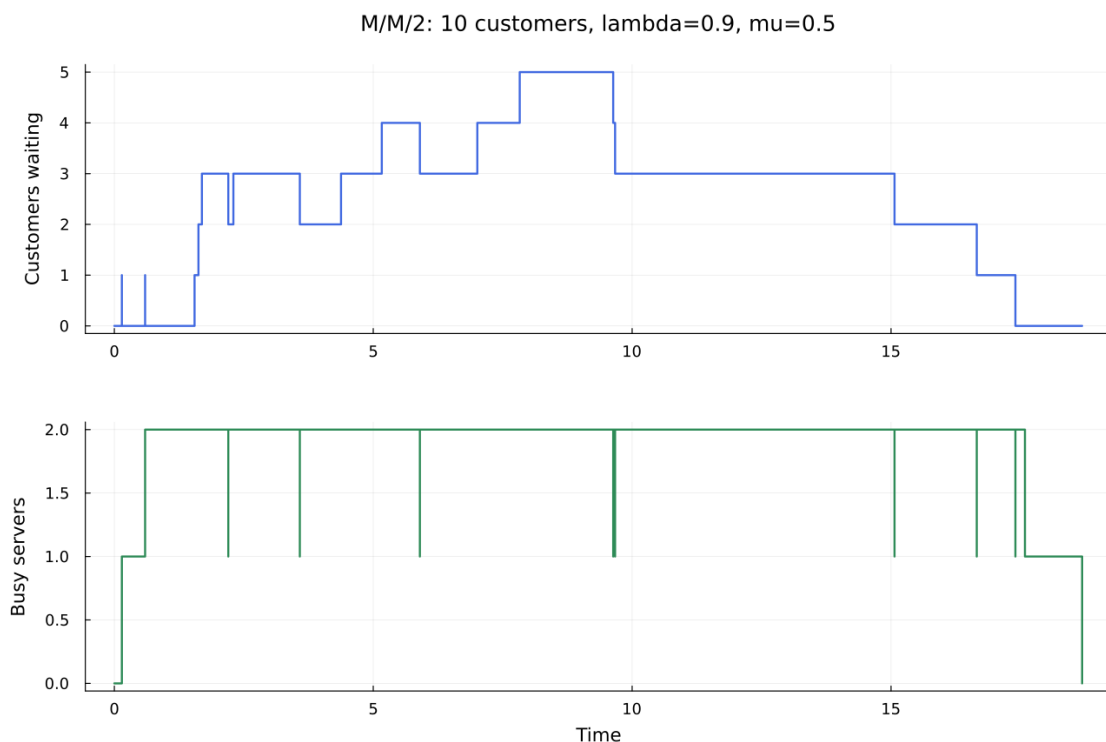


Рисунок 2: Очередь и число занятых каналов базового опыта.

На [Рисунок 2](#) ресурс никогда не обслуживает больше двух клиентов. После последнего прибытия очередь постепенно освобождается. Эта конечная фаза отличается от постоянно поступающего стационарного потока.

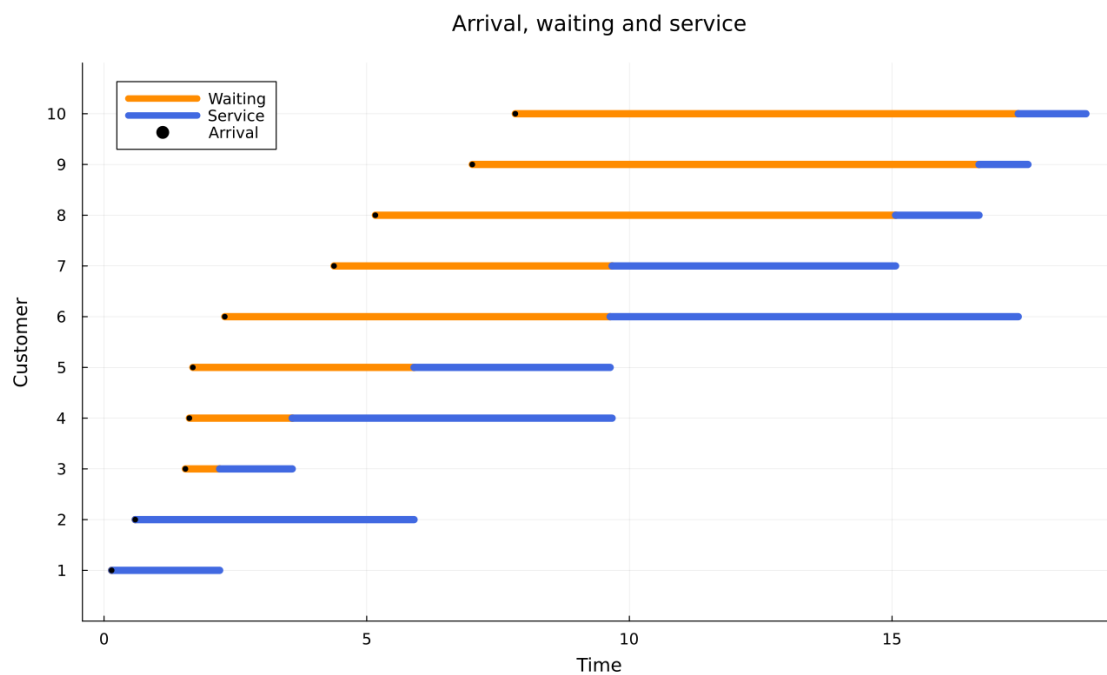


Рисунок 3: Прибытие, ожидание и обслуживание каждого из десяти клиентов.

На [Рисунок 3](#) чёрная точка отмечает прибытие, оранжевый участок — ожидание, синий — обслуживание. Начала обслуживания упорядочены по прибытию, однако окончания могут следовать в другом порядке из-за разных случайных длительностей.

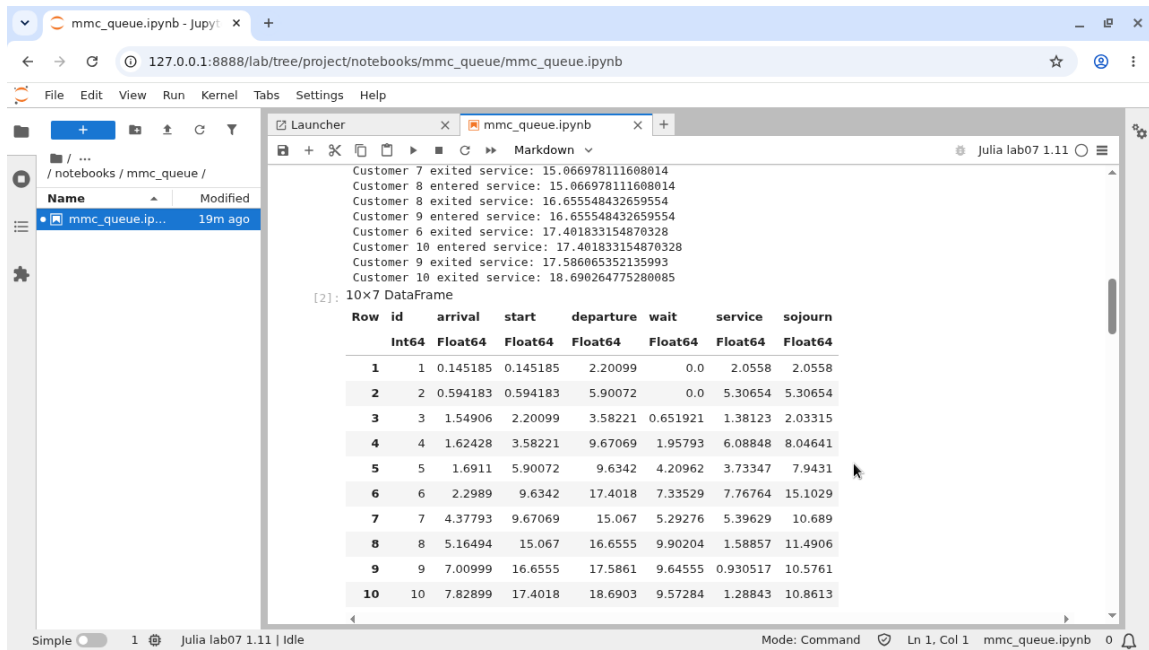


Рисунок 4: Базовая очередь в выполненном Julia-блокноте.

Блокнот (Рисунок 4) сохраняет исходные параметры, таблицу клиентов и графики. Стационарное $W_q=8,526316$ для этих интенсивностей не обязано совпадать с $4,856796$ короткого запуска из пустой системы.

4.4 Параметрическая серия и закон Литтла

Исследованы $\lambda \in \{0,3;0,6;0,9\}$ и $c \in \{1;2;3;4\}$ при $\mu=0,5$. Каждый из 12 опытов включает 20 000 клиентов; первые 2 000 исключены из клиентских оценок. Seed равен 7000 плюс номер сочетания, где λ изменяется быстрее c . Всего сохранены 240 000 строк клиентских данных.

Средние очереди и занятости считаются по времени, а не усреднением строк:

$$\bar{Q} = \frac{1}{t_1 - t_0} \sum_j Q(t_j) |I \cap [t_0, t_1]|.$$

t_0 — прибытие первого клиента после разогрева, t_1 — последнее прибытие. Клиентские времена досчитываются до обслуживания всех заявок, но интеграл состояния не включает искусственный участок освобождения системы. На конечном окне $L_q - \lambda W_q$ и $L - \lambda W$ не равны строго нулю: присутствуют граничные клиенты и отличие фактической интенсивности от заданной λ .

```
julia --project=. scripts/mmc_queue_param.jl
```

4.4.1 М/М/с: интенсивность потока и число каналов

Параметрическое дополнение §7.4. Аналитические показатели применимы при $\rho < 1$.

```
using DrWatson
@quickactivate "project"
include(sourcedir("MMCQueue.jl"))
using .MMCQueue
include(sourcedir("experiment_support.jl"))
```

4.4.1.1 План эксперимента

Каждый опыт содержит 20000 заявок. Первые 2000 исключаются из клиентских оценок; интегралы состояния считаются до последнего прибытия, без фазы слива.

```
arrival_rates = [.3, .6, .9]
capacities = [1, 2, 3, 4]
num_customers, warmup,  $\mu$  = 20_000, 2_000, .5
summaries, sampled = NamedTuple[], DataFrame[]
```

4.4.1.2 Выполнение двенадцати сочетаний

Сохраняются все времена клиентов; для обзорных траекторий — 1001 отсчёт фактического кусочно-постоянного состояния каждого календаря событий.

```
for (index, ( $\lambda$ , c)) in
enumerate(Iterators.product(arrival_rates, capacities))
    seed = 7000+index
    result = simulate_mmc(; num_customers, num_servers=c,  $\lambda$ ,  $\mu$ , seed)
    observed, theory = queue_summary(result; warmup), erlang_c( $\lambda$ ,  $\mu$ , c)
    push!(summaries, merge((experiment=index, seed=seed), observed,
        (rho=theory.rho, stable=theory.stable, theory_Wq=theory.Wq,
            theory_W=theory.W, theory_Lq=theory.Lq, theory_L=theory.L,
            theory_p_wait=theory.p_wait)))
    clients = insertcols!(result.customers, 1, :experiment=>index,
:  $\lambda$ => $\lambda$ , :c=>c)
    CSV.write(datadir("mmc_param_customers.csv"), clients;
append=index>1, writeheader=index==1)
    trace = sampled_queue(result.events,
last(result.customers.arrival))
    insertcols!(trace, 1, :experiment=>index, :  $\lambda$ => $\lambda$ , :c=>c)
    push!(sampled, trace)
end
summary = DataFrame(summaries)
traces = vcat(sampled...)
save_table("mmc_param_summary.csv", summary)
save_table("mmc_param_states.csv", traces)
select(summary,
:  $\lambda$ , :c, :rho, :stable, :Wq, :theory_Wq, :utilization, :p_wait, :theory_p_wait)
```

4.4.1.3 Среднее ожидание: имитация и формула Эрланга C

```
p_wait = plot(; xlabel="Servers c", ylabel="Mean waiting time",
title="Stationary cases: simulation and Erlang C")
for (index, λ) in enumerate(arrival_rates)
  rows = filter(row->row.λ==λ && row.stable, summary)
  color=[:royalblue,:seagreen,:darkorange][index]
  plot!(p_wait, rows.c, rows.Wq; marker=:circle, color, label="DES,
lambda=$λ")
  plot!(p_wait, rows.c, rows.theory_Wq; linestyle=:dash, color,
label="Theory, lambda=$λ")
end
savefig(p_wait, imagepath("03-plot.png"))
display(p_wait)
```

4.4.1.4 Загрузка и проверка стационарных балансов

```
stable = filter(:stable=>identity, summary)
p_busy = scatter(stable.rho, stable.utilization; xlabel="Theoretical
utilization rho",
  ylabel="Observed utilization", label="DES", color=:royalblue,
  title="Time-weighted server utilization")
plot!(p_busy, [0,1], [0,1]; color=:firebrick, linestyle=:dash,
label="y = x")
savefig(p_busy, imagepath("04-plot.png"))
display(p_busy)
select(stable,
:λ,:c,:Lq,:theory_Lq,:little_Lq_residual,:little_L_residual)
```

4.4.1.5 Неустойчивые очереди

При $\lambda \geq c\mu$ длинный конечный прогон всё ещё завершается после обслуживания всех клиентов, но стационарного Wq нет. Пустые теоретические значения не заменяются нулями.

```
p_unstable = plot(; xlabel="Time", ylabel="Customers waiting",
title="One server: overload grows the queue")
for λ in [.6,.9]
  rows = filter(row->row.λ==λ && row.c==1, traces)
  plot!(p_unstable, rows.time, rows.queue; label="lambda=$λ, c=1",
seriestype=:steppost)
end
savefig(p_unstable, imagepath("05-plot.png"))
display(p_unstable)
filter(:stable=>!, summary)
```

Таблица 3: Ожидание после разогрева во всех сочетаниях

λ	c	ρ	Wq, опыт	Wq, теория
0,3	1	0,60	3,183015	3,000000
0,6	1	1,20	3837,500125	нет
0,9	1	1,80	9690,747835	нет
0,3	2	0,30	0,208983	0,197802
0,6	2	0,60	1,152095	1,125000
0,9	2	0,90	9,128937	8,526316
0,3	3	0,20	0,020125	0,020548
0,6	3	0,40	0,147471	0,156863
0,9	3	0,60	0,567654	0,591241
0,3	4	0,15	0,000965	0,002051
0,6	4	0,30	0,027861	0,026464
0,9	4	0,45	0,103164	0,116849

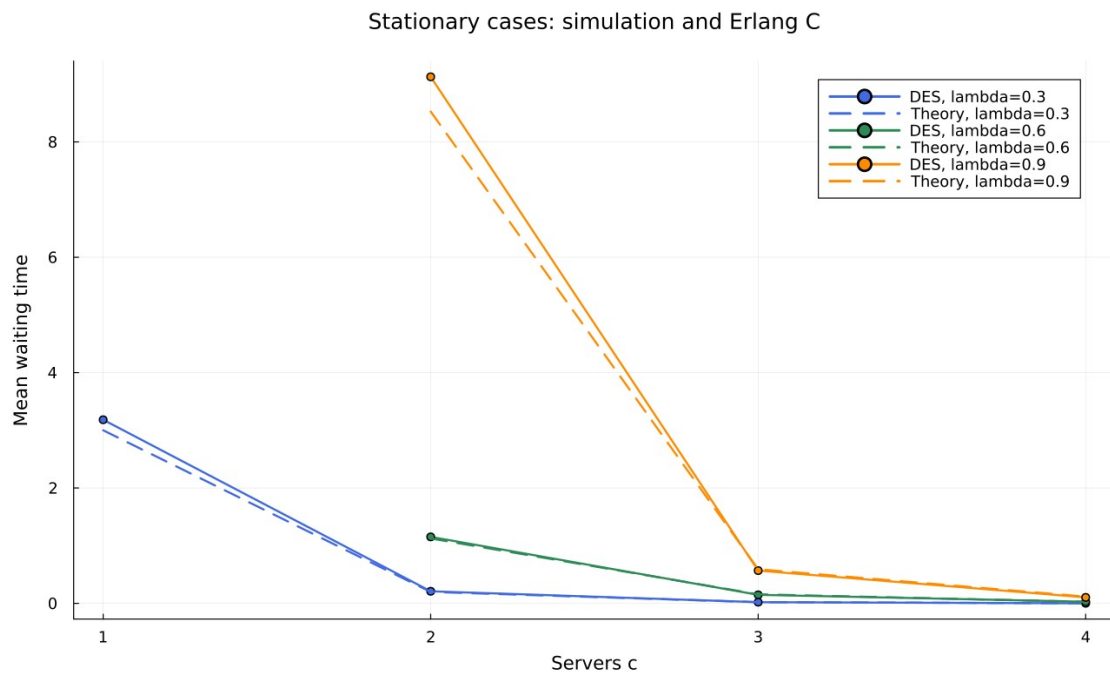


Рисунок 5: Ожидание в устойчивых случаях и формула Эрланга С.

Увеличение числа каналов резко сокращает ожидание (Рисунок 5). При $\lambda=0,9$ переход от двух к трём каналам уменьшает наблюдаемое W_q с 9,128937 до 0,567654. Это следствие снижения загрузки с 0,9 до 0,6, а не изменения распределения обслуживания.

Согласие с теорией нельзя оценивать только общим видом линий. При малой загрузке $c=4$, $\lambda=0,3$ ожидание возникает редко: в анализируемых 18 000 клиентах ожидали лишь 49. Относительное расхождение W_q здесь велико, хотя абсолютная разность около 0,0011 мала. Один длинный прогон не даёт универсальной гарантии точности для всех редких событий.

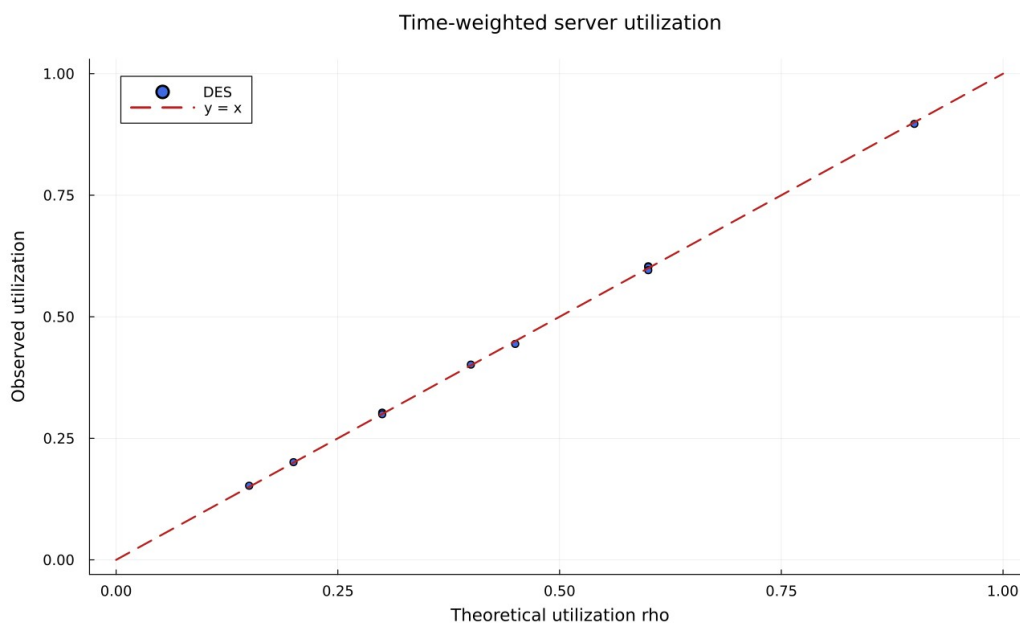


Рисунок 6: Наблюдаемая загрузка каналов и теоретическая ρ .

Загрузка (Рисунок 6) значительно устойчивее редких оценок ожидания. Для $c=2$, $\lambda=0,9$ она равна 0,896747 при теоретических 0,9; остаток $L_q - \lambda W_q$ равен 0,079802. Остаток — диагностическая величина конечного окна, не автоматическое доказательство ошибки модели.

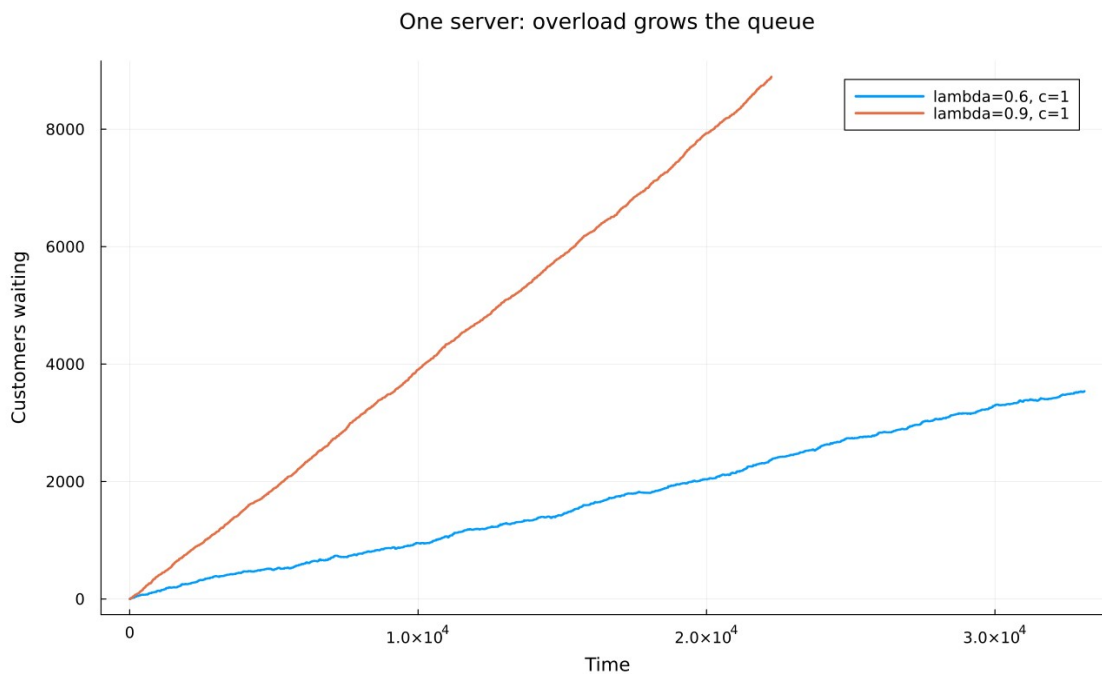


Рисунок 7: Рост очереди при перегрузке одного канала.

При $c=1$ и $\lambda=0,6$ либо 0,9 поток превышает производительность 0,5. Очередь растёт (Рисунок 7). Конечные эксперименты завершаются после прекращения поступлений и

обслуживания остатка, но это не превращает неустойчивую систему в стационарную. Теоретические поля в CSV оставлены пустыми.

summary = DataFrame(summaries)
traces = vcat(sampled...)
save_table("mmc_param_summary.csv", summary)
save_table("mmc_param_states.csv", traces)
select(summary, :lambda, :c, :rho, :stable, :Wq, :theory_Wq, :utilization, :p_wait, :theory_p_wait)

[3]: 12x9 DataFrame

Row	λ	c	ρ	stable	Wq	theory_Wq	utilization	p_wait	theory_p_wait
Float64	Int64	Float64	Bool	Float64	Float64?	Float64?	Float64	Float64	Float64?
1	0.3	1	0.6	true	3.18301	3.0	0.603821	0.605167	0.6
2	0.6	1	1.2	false	3837.5	missing	1.0	1.0	missing
3	0.9	1	1.8	false	9690.75	missing	1.0	1.0	missing
4	0.3	2	0.3	true	0.208983	0.197802	0.302832	0.147056	0.138462
5	0.6	2	0.6	true	1.15209	1.125	0.602657	0.453833	0.45
6	0.9	2	0.9	true	9.12894	8.52632	0.896747	0.847278	0.852632
7	0.3	3	0.2	true	0.0201247	0.0205479	0.200957	0.0253333	0.0246575
8	0.6	3	0.4	true	0.147471	0.156863	0.40158	0.1365	0.141176
9	0.9	3	0.6	true	0.567654	0.591241	0.59581	0.347167	0.354745
10	0.3	4	0.15	true	0.00096532	0.00205066	0.152433	0.00272222	0.00348612
11	0.6	4	0.3	true	0.0278606	0.0264641	0.299526	0.0383333	0.0370497
12	0.9	4	0.45	true	0.103164	0.116849	0.444142	0.121944	0.128534

Рисунок 8: Параметрическая М/М/с: выполненные ячейки и результаты.

Все двенадцать сочетаний доступны в таблице блокнота (Рисунок 8). Для обзорных графиков сохранено по 1001 отсчёту фактического состояния; усреднение по времени производится по исходным событиям, не по этой сетке.

5. Система Росса

5.1 Холодный резерв и критерий аварии

Одновременно работают N одинаковых машин. Ещё S машин находятся в холодном резерве: пока они не задействованы, отказов у них нет. Сломавшаяся машина немедленно заменяется резервной и поступает в ремонт. После ремонта она возвращается в резерв. r одинаковых ремонтников обслуживают не более r неисправных машин одновременно; остальные ожидают в общей очереди.

В исходном опыте $N=10$, $S=3$, $r=1$. $\text{Exponential}(100)$ означает среднее время работы одной машины 100 часов, а $\text{Exponential}(1)$ — средний ремонт 1 час. Интенсивности равны соответственно 0,01 и 1 в час. Пока система работоспособна, суммарная интенсивность отказов $N/100=0,1$ в час. Следовательно, ремонт в среднем намного быстрее интервала между отказами.

Если k машин неисправны, исправных $N+S-k$. При $k=S$ исправных ещё N , все они работают, но резерва нет. Авария возникает при следующем отказе: $k=S+1$, исправных становится $N-1$. Остановка уже при нулевом резерве была бы другой моделью с более ранним временем аварии.

5.2 Процессы машин, резерв и ремонтный ресурс

В листинге 2 приведён жизненный цикл из `src/RossRepair.jl`. Ожидание `timeout(env, Inf)` соответствует холодному резерву. Перехватывается только `InterruptedException`, используемое для активации машины; другие исключения не скрываются. Составное событие даёт немедленный результат запроса резерва.

```
while true
  try
    @yield timeout(env, Inf)
  catch exception
    exception isa ConcurrentSim.InterruptException || rethrow()
  end
  @yield timeout(env, rand(rng, F))
  record!(monitor, env, "failure", id; broken_delta=1)
  get_spare = take!(spares)
  @yield get_spare | timeout(env)
  if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
  else
    record!(monitor, env, "crash", id)
    throw(StopSimulation("No more spares!"))
  end
  @yield request(repair)
  record!(monitor, env, "repair_start", id; busy_delta=1)
  @yield timeout(env, rand(rng, G))
  record!(monitor, env, "repair_complete", id;
    broken_delta=-1, busy_delta=-1)
  @yield unlock(repair)
  @yield put!(spares, active_process(env))
end
```

Листинг 2 — Жизненный цикл машины, `src/RossRepair.jl`.

Замена `r=1` на большее значение меняет ёмкость `Resource`, а не вводит искусственное ускорение одной ремонтной операции. Каждый ремонт по-прежнему имеет независимую экспоненциальную длительность со средним 1 час.

Монитор сначала интегрирует старое состояние на интервале от предыдущего события, затем меняет счётчики. Для времени аварии T рассчитываются

$$U = \frac{1}{rT} \int_0^T b(t) dt, \bar{Q} = \frac{1}{T} \int_0^T (k(t) - b(t)) dt,$$

где $b(t)$ — число занятых ремонтников. Простое среднее столбца `busy` по строкам CSV было бы неверным: длинные и короткие интервалы имели бы одинаковый вес.

5.3 Аналитическая проверка поглощающей цепью

Из-за отсутствия памяти экспоненциальных распределений число неисправных машин k является достаточным состоянием. До аварии $k \in \{0, \dots, S\}$; состояние $S+1$ поглощающее. Интенсивности переходов:

$$q_{k,k+1} = a = \frac{N}{F}, q_{k,k-1} = b_k = \frac{\min(k, r)}{G}, q_{k,k} = -(a + b_k),$$

где F и G — средние времена работы и ремонта. Переход из S в $S+1$ не входит отдельным столбцом в транзиентную матрицу Q , но его интенсивность остаётся в диагонали. Иначе цепь не могла бы поглощаться.

Вектор средних времён до аварии m удовлетворяет $Qm = -1$. Ожидаемые времена пребывания в состояниях из начального $k=0$ образуют вектор v , определяемый системой $(-Q)^T v = e_0$. Тогда $E[T] = m_0 = \sum v_k$, а аналитические загрузка и очередь получаются взвешиванием $\min(k, r)$ и $\max(k-r, 0)$ по v . Используются решения линейных систем, а не явное обращение матрицы.

```
failure = N/failure_mean
Q = zeros(S+1, S+1)
for k in 0:S
    repair = min(k, repairers)/repair_mean
    Q[k+1, k+1] = -(failure+repair)
    k < S && (Q[k+1, k+2] = failure)
    k > 0 && (Q[k+1, k] = repair)
end
times = -Q \ ones(S+1)
start = zeros(S+1); start[1] = 1.0
occupation = transpose(-Q) \ start
```

Листинг 3 — Транзиентная матрица и средние времена, `src/RossRepair.jl`.

Для одного ремонтника результат независимо проверяется рекуррентной формулой

$$E[T] = \frac{1}{a} \sum_{k=0}^S \sum_{j=0}^k \left(\frac{b}{a} \right)^j, a = N/F, b = 1/G.$$

Для $N=10$, $S=3$, $r=1$ получено $E[T]=12\,340$ часов, $U=0,09967585$, средняя очередь $0,01053485$. Это аналитика конкретной модели до первого поглощения, а не произвольное сравнение с обычной бесконечной $M/M/1$.

5.4 Пять базовых прогонов

Сохранены $N=10$, $S=3$, $r=1$, $F=100$, $G=1$ и $RUNS=5$. В тексте исходника практикума объявлен $SEED=150$, но в вычислении фактически применяется $StableRNG(42)$. Здесь воспроизведён именно действительный генератор: один поток последовательно используется во всех пяти прогонах, без сброса seed перед каждым из них.

```
julia --project=. scripts/ross_repair.jl
```

5.4.1 Система Росса: резерв и один ремонтник

Базовые параметры §7.3.3: $N=10$, $S=3$, пять запусков, средние времена 100 и 1 час.

```
using DrWatson
@quickactivate "project"
include(sourcedir("RossRepair.jl"))
using .RossRepair
using StableRNGs
include(sourcedir("experiment_support.jl"))
```

5.4.1.1 Пять последовательных прогонов исходного опыта

В исходном коде фактически используется $StableRNG(42)$, хотя $SEED=150$ объявлен отдельно и не применяется. Сохраняется действительная последовательность $rng=42$.

```
rng = StableRNG(42)
runs, trajectories = NamedTuple[], DataFrame[]
for run_id in 1:5
    result =
simulate_repair
r(N=10,S=3,repairers=1,failure_mean=100.0,repair_mean=1.0;
    rng, verbose=true)
    push!(runs, merge((run=run_id, seed=42), result.summary))
    push!(trajectories, insertcols!(result.events, 1, :run=>run_id))
end
results, events = DataFrame(runs), vcat(trajectories...)
save_table("ross_runs.csv", results)
save_table("ross_events.csv", events)
```

```
println("Average crash time: ", mean(results.crash_time))
results
```

5.4.1.2 Аналитическое среднее время и мониторинг

Поглощение — первое достижение $S+1$ неисправных машин, а не просто нулевой резерв. Загрузка и очередь усредняются по времени, не по числу записей событий.

```
theory = analytic_repair()
estimate = mean_interval(results.crash_time)
comparison = DataFrame([(runs=5, mean_time=estimate.mean,
se=estimate.se,
ci_low=estimate.ci_low, ci_high=estimate.ci_high,
theory_time=theory.mean_time,
utilization=sum(results.area_busy)/sum(results.crash_time),
theory_utilization=theory.utilization,
mean_queue=sum(results.area_queue)/sum(results.crash_time),
theory_queue=theory.mean_queue)])
save_table("ross_comparison.csv", comparison)
save_table("ross_occupation_theory.csv", theory.occupation)
comparison
```

5.4.1.3 Полная траектория первого запуска

```
first_run = filter(:run=>==(1), events)
p_full = repair_plot(first_run,10)
savefig(p_full,imagepath("06-plot.png"))
display(p_full)
```

5.4.1.4 Исчерпание резерва перед аварией

```
t_crash = last(first_run.time)
tail_start = max(0.0,t_crash-60)
prior = max(1,searchsortedlast(first_run.time,tail_start))
p_tail = repair_plot(first_run[prior:end,:],10; title="Last 60 hours:
exhaustion and crash")
savefig(p_tail,imagepath("07-plot.png"))
display(p_tail)
```

5.4.1.5 Разброс времени до падения

```
p_runs = bar(results.run,results.crash_time; color=:royalblue,
label="Five DES runs",
xlabel="Run", ylabel="Crash time (h)", title="Five observations
are not a precise mean")
hline!(p_runs,[theory.mean_time]; color=:firebrick, linestyle=:dash,
label="CTMC mean")
savefig(p_runs,imagepath("08-plot.png"))
display(p_runs)
theory.occupation
```

Таблица 4: Исходная последовательность пяти прогонов

Прогон	Время аварии, ч	Загрузка ремонтника	Средняя очередь
1	12 715,718225	0,101820	0,012732
2	37 335,535676	0,099657	0,011228
3	30 844,626678	0,100806	0,010236
4	1 601,252491	0,106319	0,014589
5	824,104871	0,110306	0,017696

Пять времён (Таблица 4) совпали с контрольной последовательностью официального примера ConcurrentSim. Среднее равно 16 664,247588 часов, стандартная ошибка — 7 489,259406 часа. Приближённый t-интервал 95% после ограничения нижней границы нулём составляет [0;37 457,765208]. Такой широкий интервал подчёркивает, что пяти наблюдений недостаточно для точной оценки математического ожидания. Он не является точным распределительным результатом для асимметричного времени аварии.

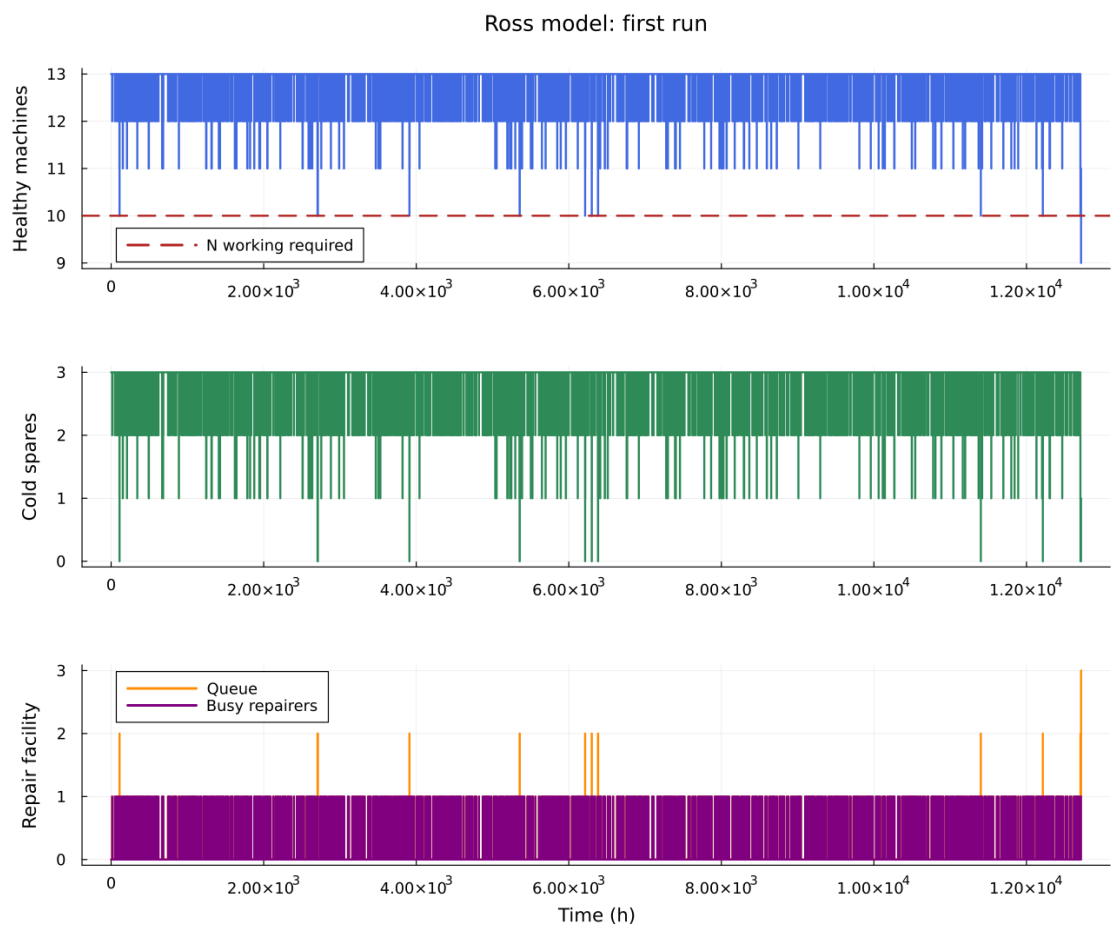


Рисунок 9: Исправные машины, холодный резерв и ремонт на полной траектории.

Большую часть первого прогона машины исправны и ремонтная очередь пуста (Рисунок 9). Сжатие 12 715 часов на одну ось скрывает подробности редкого аварийного эпизода, поэтому дополнительно показан его конец.

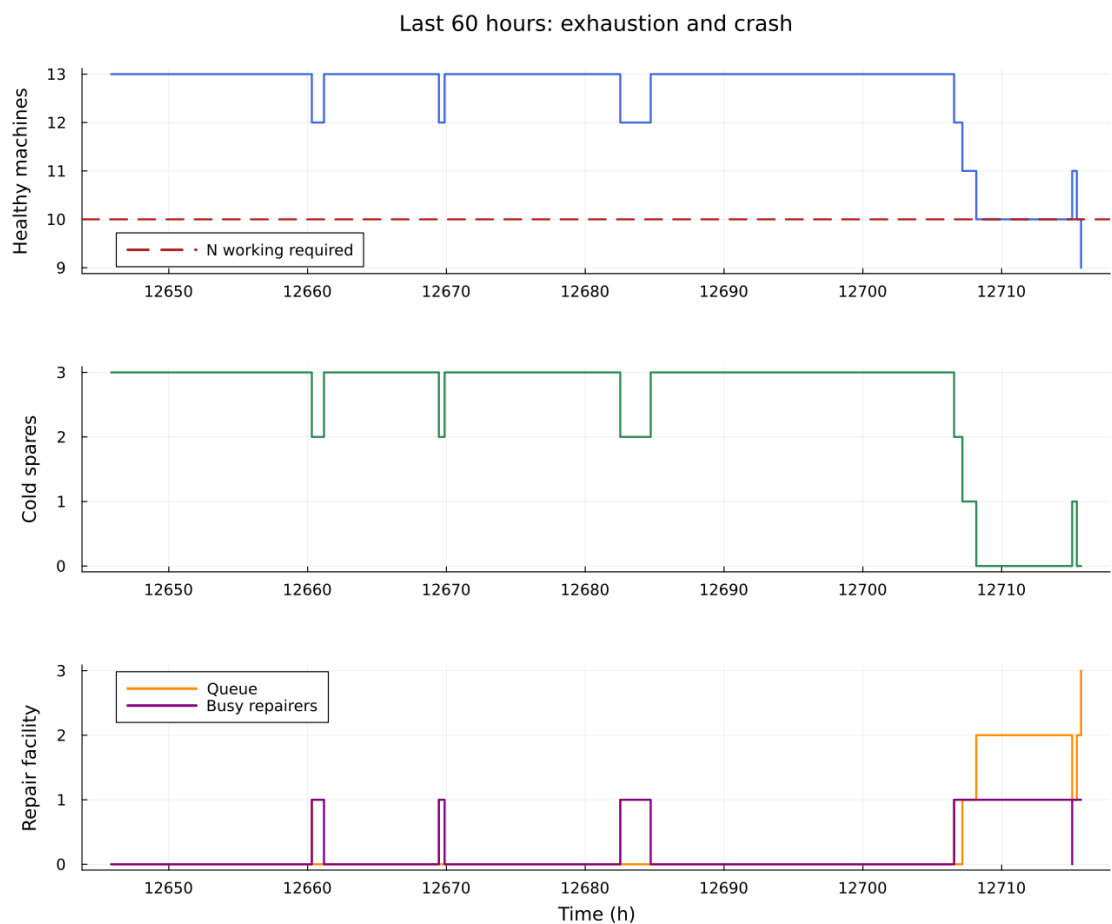


Рисунок 10: Последние 60 часов первого прогона перед аварией.

На [Рисунок 10](#) исправных становится меньше N только в последнем переходе. Нулевой резерв до него ещё допускает работу всех десяти машин. Число работающих и резервных машин определяется из текущего числа исправных, а ёмкость ремонта никогда не превышает.

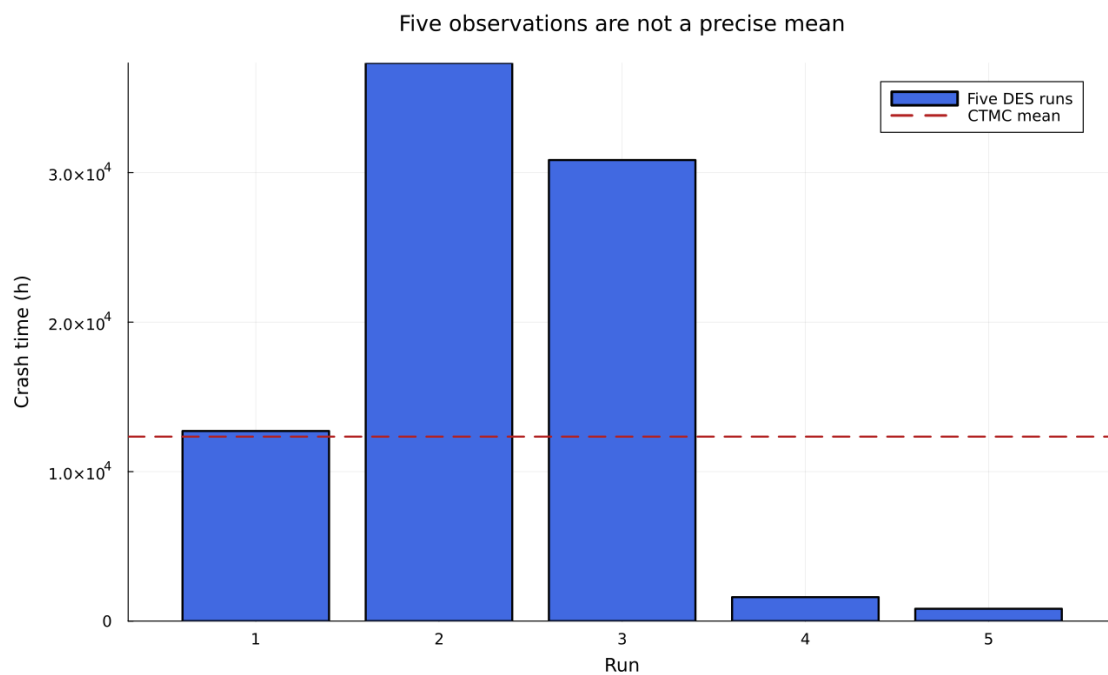


Рисунок 11: Разброс пяти времён аварии относительно аналитического среднего.

Рисунок 11 показывает сильную вариативность времени до редкой аварии. Для сопоставления загрузки и очереди с отношением аналитических ожидаемых интегралов используются объединённые оценки: $\Sigma \text{area_busy} / (r \Sigma T)$ и $\Sigma \text{area_queue} / \Sigma T$. Они равны 0,10064594 и 0,01121856. Среднее пяти индивидуальных отношений U не эквивалентно этому расчёту.

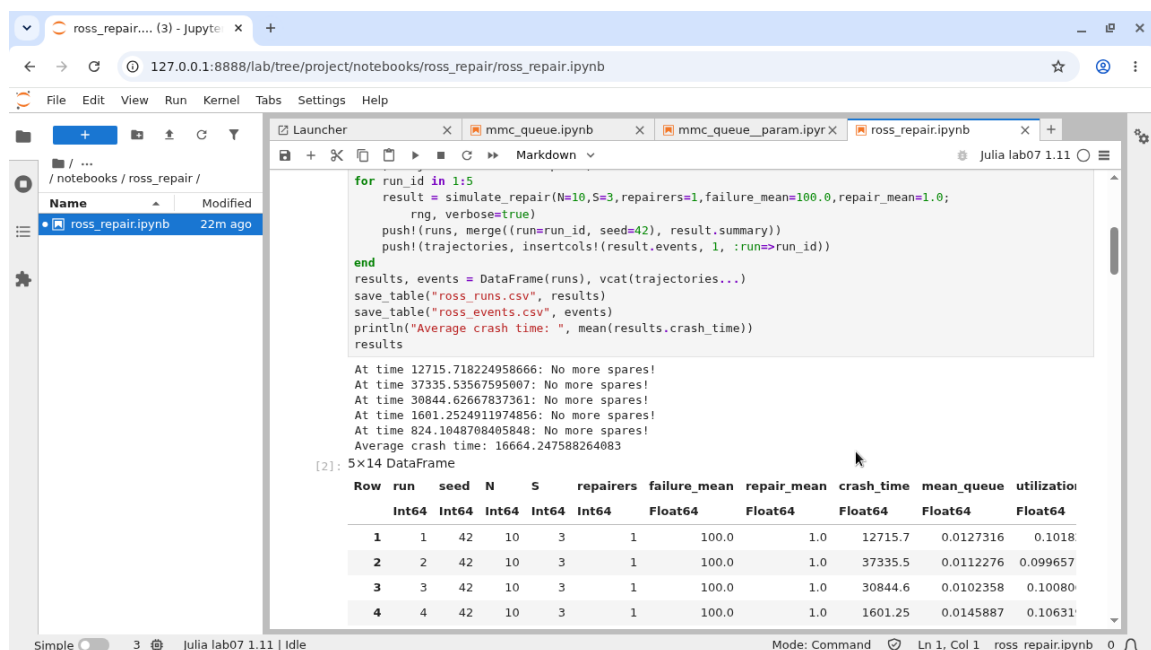


Рисунок 12: Базовая система Росса в выполненном блокноте.

В блокноте (Рисунок 12) сохранены пять результатов и таблица ожидаемых времён пребывания в состояниях аналитической цепи.

5.5 Несколько ремонтников и изменение числа машин

В параметрическом эксперименте $N \in \{5;10;20\}$, $r \in \{1;2;3\}$, $S=3$, $F=100$, $G=1$. Число повторов методичкой не задано; выбрано по 200 независимых прогонов на сочетание, всего 1800. Для сочетания i и повтора j $seed=710000+1000i+j$. Каждый прогон использует ту же процессную модель ConcurrentSim; отключено лишь сохранение большого журнала событий.

Для среднего времени приведены стандартная ошибка $s/\sqrt{200}$ и приближённый 95%-й t-интервал. Это интервалы Монте-Карло, а не жёсткие границы будущих отказов. Каждый интервал относится к отдельному сочетанию; совместная 95%-я гарантия для всех девяти строк не заявляется.

```
julia --project=. scripts/ross_repair_param.jl
```

5.5.1 Система Росса: несколько ремонтников и разное число машин

Дополнительные задания §7.3.4 и параметрическая серия §7.4.

```
using DrWatson
@quickactivate "project"
include(scrdir("RossRepair.jl"))
```

```
using .RossRepair
include(sourcedir("experiment_support.jl"))
```

5.5.1.1 План независимых повторов

```
machine_counts, repairer_counts = [5,10,20], [1,2,3]
spares, replications = 3, 200
failure_mean, repair_mean = 100.0, 1.0
replicates, comparisons = NamedTuple[], NamedTuple[]
```

5.5.1.2 Моделирование и решение поглощающей марковской цепи

Для каждого сочетания — отдельные независимые seed. Все траектории выполняются теми же процессами ConcurrentSim; отключается только сохранение длинного журнала.

```
for (index,(N,r)) in
enumerate(Iterators.product(machine_counts,repairer_counts))
    rows = NamedTuple[]
    for replication in 1:replications
        seed = 710000+1000*index+replication
        result = simulate_repair(; N, S=spares, repairers=r,
failure_mean, repair_mean,
            seed, record_events=false)
        push!
    (rows
,merge((experiment=index,replication=replication,seed=seed),result.sum
mary))
    end
    append!(replicates,rows)
    sample = DataFrame(rows)
    estimate = mean_interval(sample.crash_time)
    theory =
analytic_repair(;N,S=spares,repairers=r,failure_mean,repair_mean)
    push!(comparisons,
(experiment=index,N=N,S=spares,repairers=r,replications=replications,
mean_tim
e=estimate.mean,se=estimate.se,ci_low=estimate.ci_low,ci_high=estimate
.ci_high,
theory_tim
e=theory.mean_time,relative_error=(estimate.mean/theory.mean_time-1),
ci_contains_theor
y=estimate.ci_low<=theory.mean_time<=estimate.ci_high,
utilizatio
n=sum(sample.area_busy)/(r*sum(sample.crash_time)),theory_utilization=
theory.utilization,
```

```

mean_queue
e=sum(sample.area_queue)/sum(sample.crash_time),theory_queue=theory.mean_queue))
  println("N=$N, repairers=$r, runs=$replications, mean=$(round(estimate.mean;digits=2)) h")
end
results, comparison = DataFrame(replicates),DataFrame(comparisons)
save_table("ross_param_replicates.csv",results)
save_table("ross_param_comparison.csv",comparison)
select
t(comparison,:N,:repairers,:mean_time,:se,:ci_low,:ci_high,:theory_time,:relative_error,:ci_contains_theory)

```

5.5.1.3 Время до аварии: среднее, интервал и аналитика

```

p_time=plot(;xlabel="Working machines N",ylabel="Mean crash time (h), log scale",
  yscale=:log10,title="Cold spares S=3: effect of repair capacity")
for (index,r) in enumerate(repairer_counts)
  rows=filter(:repairers==r,comparison)
  color=[:royalblue,:seagreen,:darkorange][index]
  plot!(p_time,rows.N,rows.mean_time; yerror=(rows.mean_time-rows.ci_low,rows.ci_high-rows.mean_time),
    color,marker=:circle,label="DES, r=$r; 95% CI")
  plot!
  (p_time,rows.N,rows.theory_time;color,linestyle=:dash,label="CTMC, r=$r")
end
savefig(p_time,imagepath("09-plot.png"))
display(p_time)

```

5.5.1.4 Интегральная загрузка и очередь ремонта

```

p_util=plot(;xlabel="Working machines N",ylabel="Utilization per repairer",title="Time-weighted repair load")
p_queue=plot(;xlabel="Working machines N",ylabel="Mean repair queue",title="Waiting machines")
for (index,r) in enumerate(repairer_counts)
  rows=filter(:repairers==r,comparison)
  color=[:royalblue,:seagreen,:darkorange][index]
  plot!
  (p_util,rows.N,rows.utilization;color,marker=:circle,label="DES r=$r")
  plot!
  (p_util,rows.N,rows.theory_utilization;color,linestyle=:dash,label="CTMC r=$r")
  plot!
  (p_queue,rows.N,rows.mean_queue;color,marker=:circle,label="DES r=$r")
  plot!
  (p_queue,rows.N,rows.theory_queue;color,linestyle=:dash,label="CTMC

```

```

r=$r")
end
p_monitor=plot(p_util,p_queue;layout=(1,2),size=(1200,600))
savefig(p_monitor,imagepath("10-plot.png"))
display(p_monitor)
selec
t(comparison,:N,:repairers,:utilization,:theory_utilization,:mean_queue,
,:theory_queue)

```

5.5.1.5 Проверка утверждения о числе резервных машин

Это аналитическое дополнение, не ещё одна имитационная серия.

```

reserve_chec
k=DataFrame( [(N=10,S=S,repairers=1,mean_time=analytic_repair(S=S).mean_time,
recurrence=one_repair_mean(10,S,100.0,1.0)) for S in 0:5])
save_table("ross_reserve_analytic.csv",reserve_check)
reserve_check

```

Таблица 5: Двести независимых прогонов для каждого сочетания

N	r	Среднее, ч	95%-й интервал, ч	Аналитика, ч	Откл., %
5	1	187 130,07	[156 084,66;218 175,48]	177 280	+5,56%
10	1	12 102,43	[10 376,68;13 828,18]	12 340	-1,93%
20	1	985,02	[841,35;1128,69]	970	+1,55%
5	2	700 789,78	[599 640,22;801 939,34]	690 080	+1,55%
10	2	52 858,15	[45 406,95;60 309,35]	46 540	+13,58%
20	2	3331,38	[2865,76;3797,00]	3395	-1,87%
5	3	1 003 502,75	[852 486,56;1 154 518,93]	1 026 480	-2,24%
10	3	67 627,88	[57 439,49;77 816,27]	68 640	-1,47%
20	3	4749,87	[4050,57;5449,16]	4920	-3,46%

Все девять интервалов в этой серии накрыли соответствующее аналитическое ожидание (Таблица 5). Это наблюдаемый результат, а не требование к любой будущей выборке. Наибольшее относительное отклонение среднего составило +13,58% при N=10 и r=2; стандартная ошибка там равна 3778,58 часа, поэтому точечное расхождение не противоречит приведённому интервалу.

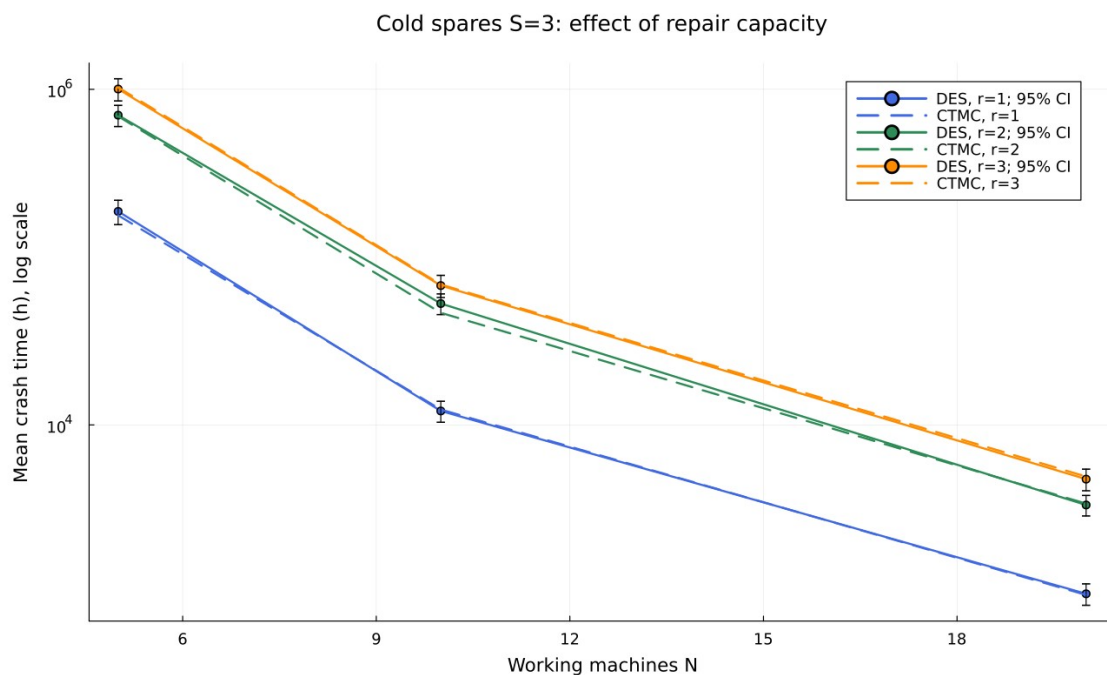


Рисунок 13: Влияние числа машин и ремонтников на среднее время аварии.

При увеличении N суммарный поток отказов N/F растёт, а S и ремонтная мощность остаются прежними, поэтому система теряет резерв быстрее. Дополнительные ремонтники уменьшают вероятность накопления $S+1$ отказов. Логарифмическая ось на [Рисунок 13](#) нужна из-за разных порядков времени, а отрезки показывают интервалы среднего, не разброс отдельных прогонов.

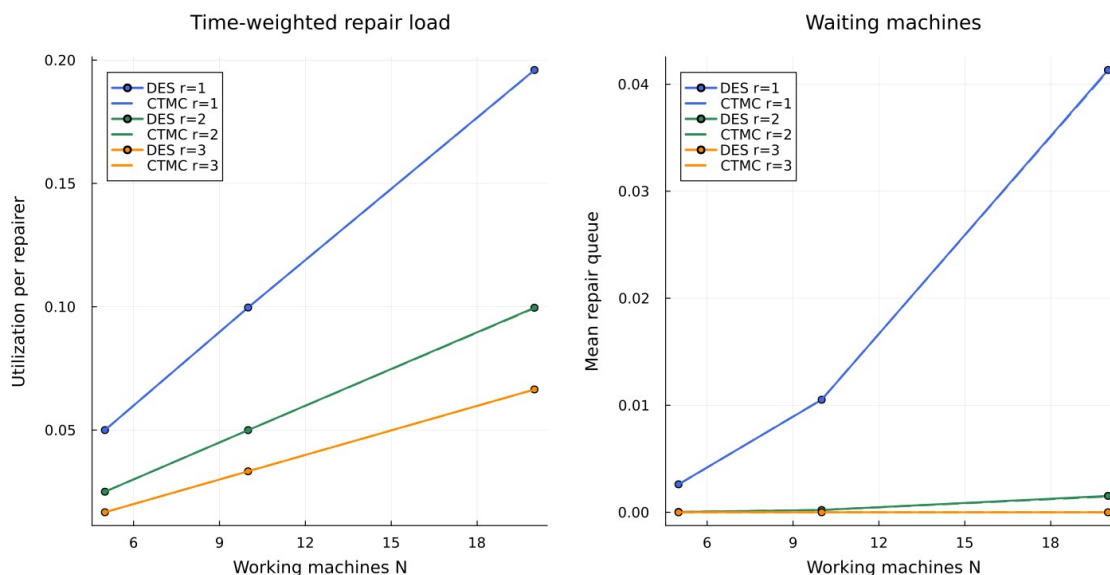


Рисунок 14: Загрузка ремонтников и очередь: имитация и аналитика.

[Рисунок 14](#) сопоставляет объединённые интегральные оценки с поглощающей цепью. При $r=3$ и $S=3$ очередь до аварии тождественно пуста: каждая из максимум трёх

неисправных машин сразу получает ремонтника. Четвёртый отказ уже завершает эксперимент и не создаёт положительного по длительности участка ожидания. Нулевая очередь здесь не означает нулевую вероятность аварии.

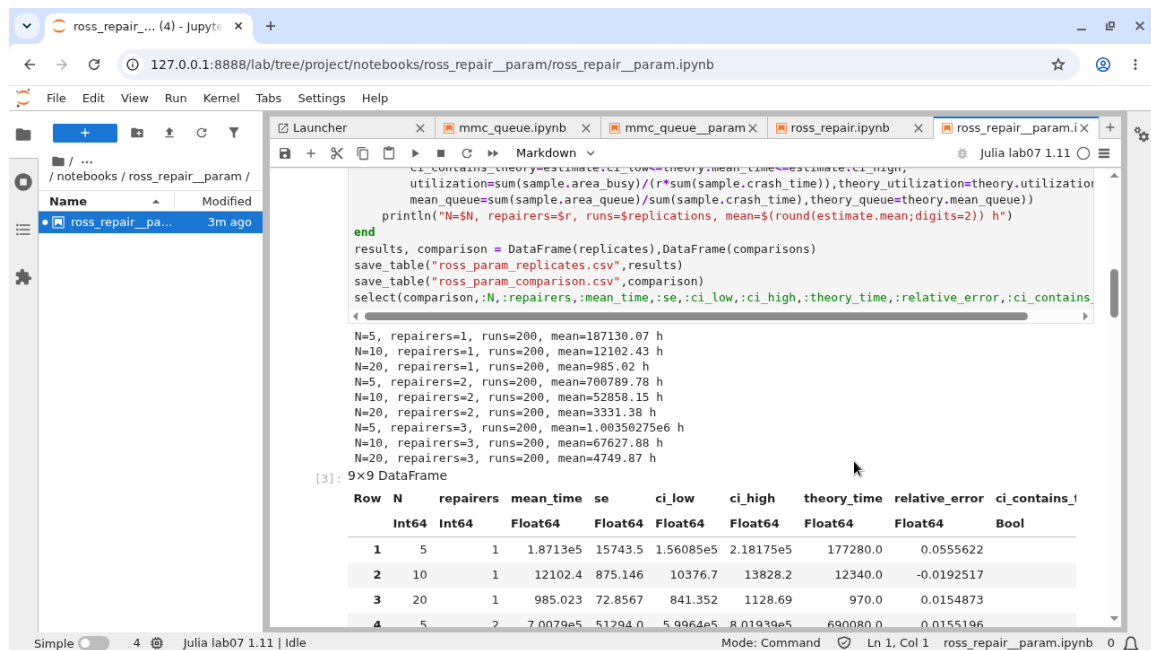


Рисунок 15: Параметрическая система Росса: результаты 1800 прогонов.

Блокнот (Рисунок 15) содержит таблицы оценок, интервалы, аналитику и графики. Отдельная аналитическая таблица меняет S от 0 до 5 при $N=10$ и $r=1$. Этот расчёт не выдаётся за дополнительную имитационную серию: он служит проверкой влияния резерва и рекуррентной формулы.

5.6 Уточнения к примеру практикума

Средние времена 100 и 1 час нельзя трактовать как интенсивности. При исходных параметрах ремонт занимает в среднем 1 час, а отказы всего работающего парка разделены в среднем 10 часами. Поэтому качественное утверждение о медленном ремонте относительно отказов не соответствует числам данного примера.

Увеличение резерва до $S=5$ при одном ремонтнике даёт аналитически $E[T]=1\,234\,560$ часов, а не несколько сотен часов. Значение проверено матричным решением и независимой рекуррентной формулой. Зависимость сильно нелинейна: пока ремонт успевает устранять обычные отказы, авария требует редкой последовательности их накопления.

Объявление SEED=150 при фактически используемом 42 описано явно. Критерий аварии уточнён как переход к N-1 исправным машинам. Корректные операции request/unlock сохранены по актуальному API; логика резервирования и ремонта не заменена другой моделью.

6. Литературные исходники и блокноты

Файл `scripts/tangle.jl` создаёт для каждого из четырёх литературных сценариев чистый Julia-код, исполняемый QMD и IPYNB. Для отчёта также создаётся вариант `*-report.qmd` с обычными Julia-листингами. Это позволяет включить весь сценарий в тематический раздел, не перезапуская тысячи моделирований при каждой правке текста отчёта.

```
Literate.script(source, scriptsdir(name); name, credit=false)
Literate.notebook(source, projectdir("notebooks", name); name,
    execute=false, credit=false, postprocess=nb -> begin
    nb["metadata"]["kernel_spec"] = Dict("language"=>"julia",
        "name"=>"julia-lab07-1.11",
        "display_name"=>"Julia lab07 1.11")
    nb
end)
Literate.markdown(source, dest; name,
    flavor=Literate.QuartoFlavor(), credit=false)
```

Листинг 4 — Получение производных форматов, `scripts/tangle.jl`.

Первоначальный `execute=false` относится только к созданию структуры IPYNB. После этого `make notebooks` действительно выполняет каждый блокнот через ядро Julia. Сохраняются счётчики выполнения и выходы всех кодовых ячеек. Команда `make docs` исполняет QMD и создаёт HTML-документацию. Все четыре `*-report.qmd` включены непосредственно в соответствующие разделы данного отчёта, а не заменены вручную выдержками.

Таблица 6: Проверка выполненных Julia-блокнотов

Блокнот	Кодовых ячеек	Счётчики	Ошибки
<code>mmc_queue.ipynb</code>	6	1–6	0
<code>mmc_queue__param.ipynb</code>	6	1–6	0
<code>ross_repair.ipynb</code>	6	1–6	0

Блокнот	Кодовых ячеек	Счётчики	Ошибки
ross_repair__param.ipynb	6	1–6	0

В каждом блокноте проверены все шесть кодовых ячеек (Таблица 6): счётчики не пусты, исключений выполнения нет, таблицы и графики сохранены.

```

dakuokkonen@lab07:/workspace/labs/lab07/project$ julia --project=. scripts/ross_repair.jl
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083
dakuokkonen@lab07:/workspace/labs/lab07/project$ wc -l data/mmc_param_customers.csv data/ross_param_replic
ates.csv
 240001 data/mmc_param_customers.csv
   1801 data/ross_param_replicates.csv
 241802 total
dakuokkonen@lab07:/workspace/labs/lab07/project$

```

Рисунок 16: Командное выполнение сценариев и проверка результатов.

Вывод запуска (Рисунок 16), содержимое CSV и выходы блокнотов проверяются отдельно: наличие файла без результата выполнения не считается завершённым экспериментом.

7. Проверки и ограничения

test/runtests.jl содержит 221 проверку в четырёх группах. Для очереди проверены причинный порядок $\text{arrival} \leq \text{start} \leq \text{departure}$, FIFO, ограничения ресурса, отсутствие отрицательной очереди, обслуживание всех заявок, повторяемость seed и тождества площадей под траекториями. Для стационарной теории проверены M/M/1, Эрланг C и запрет стационарных оценок при перегрузке.

Для системы Росса проверены все пять контрольных времён, сохранение общего числа машин, работа N машин до аварии, saracity ремонтников и временные интегралы.

Отдельно проверены $S=0$, $N=1$ и $r=1\dots 3$. Аналитика сверена с рекуррентной формулой на 24 сочетаниях N и S ; проверены положительность времён и остаток Q_{m+1} .

```
@test all(c.arrival .<= c.start .<= c.departure)
@test issorted(c.start)
@test all(0 .<= e.busy .<= 2)
@test isapprox(sum(c.wait),
    sum(diff(e.time).*e.queue[1:end-1])); rtol=1e-12)
@test analytic_repair().mean_time ≈ 12340
@test analytic_repair(N=5,S=0).mean_time ≈ 100/5
@test analytic_repair(repairers=3).mean_queue == 0
```

Листинг 5 — Фрагменты независимых проверок, *test/runtests.jl*.

```
bash
dakuokkonen@lab07:/workspace/labs/lab07/project$ make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
M/M/c: process and resource invariants | 20 20 1.9s
Test Summary: | Pass Total Time
Erlang C: independent M/M/1 formulas | 11 11 1.9s
Test Summary: | Pass Total Time
Ross: reference sequence and event monitoring | 63 63 4.3s
Test Summary: | Pass Total Time
Ross: absorbing CTMC and independent recurrence | 127 127 0.0s
dakuokkonen@lab07:/workspace/labs/lab07/project$
```

Рисунок 17: Результат выполнения тестов Julia.

Тесты на [Рисунок 17](#) подтверждают программные инварианты, но не устраняют статистическую погрешность. В М/М/с использован один длинный прогон на сочетание; редкое ожидание при низкой загрузке оценено менее точно. В модели Росса 200 повторов существенно содержательнее пяти, однако интервалы среднего остаются приближёнными. Аналитические формулы опираются на независимые экспоненциальные времена и холодный резерв; при старении, общих причинах отказов или отказах резервных машин эта цепь уже не описывает систему без изменения состояний и переходов.

8. Выводы

Две модели реализованы средствами одного событийного механизма, но отвечают на разные вопросы. М/М/с показывает накопление ожидания при приближении входного потока к пропускной способности. Система Росса связывает число резервных машин и ремонтную мощность со временем до потери работоспособности.

Для М/М/с выполнены исходные десять клиентов и двенадцать параметрических опытов. Разделены короткий переходный запуск, устойчивые режимы и перегрузка. При $\lambda=0,9$ добавление третьего канала уменьшило наблюдаемое ожидание с 9,128937 до 0,567654. Средние по состояниям получены интегрированием межсобытийных интервалов; конечные расхождения закона Литтла сохранены как диагностические результаты.

В системе Росса воспроизведены пять исходных времён аварии, построены траектории исправных и резервных машин, загрузки и очереди ремонта. Параметрическая серия содержит 1800 независимых прогонов для девяти сочетаний числа машин и ремонтников. Сравнение выполнено с поглощающей марковской цепью, дающей $E[T]=12\,340$ часов для базовых условий. Показано, почему отсутствие свободного резерва ещё не равно аварии и почему среднее пяти наблюдений нельзя считать точным математическим ожиданием.

Четыре литературных сценария согласованы с чистыми Julia-файлами, выполненными блокнотами и QMD-документацией. Полученные данные, рисунки, листинги и аналитические ограничения связаны в одном исследовании.

Источники

1. Королькова А. В., Кулябов Д. С. *Имитационное моделирование. Практикум*. Глава 7 «Дискретно-событийное моделирование», с. 209–219.
2. ConcurrentSim.jl. *M/M/c queueing system*. <https://juliadynamics.github.io/ConcurrentSim.jl/stable/examples/mmc/>.
3. ConcurrentSim.jl. *A discrete event example (Ross)*. <https://juliadynamics.github.io/ConcurrentSim.jl/stable/examples/ross/>.
4. ConcurrentSim.jl. *API*. <https://juliadynamics.github.io/ConcurrentSim.jl/stable/api/>.
5. Literate.jl. *File formats and output*. <https://fredriekre.github.io/Literate.jl/v2/>.

6. Quarto. *Scientific and technical publishing*. <https://quarto.org/docs/>.